University of Essex

Online

Introduction to Object-Oriented Software Architecture

The OO software architecture can be defined from different perspectives. Some of the definitions of OO software architecture are presented below:

An OO software architecture organises a system into interacting objects (classes/instances) that encapsulate data and behaviour, promoting modularity and reusability. Example: A banking system with Account, Transaction, and Customer classes, each handling specific responsibilities.

An architecture leveraging OOP design patterns (e.g., Factory, Observer) to solve recurring structural problems in scalable systems. Example: Using the Observer pattern in a stock market app to notify users (observers) when stock prices (subject) change.

A hierarchical structure where each layer (e.g., UI, Business Logic, Data) is built using OOP principles for separation of concerns. Example: A web app with: Presentation Layer: UserInterface class (React/Vue), Business Layer: OrderService class (Java/Python), Data Layer: DatabaseRepository class (SQL/NoSQL).

A system decomposed into reusable OO components (e.g., libraries, services) communicating via well-defined interfaces. Example: An e-commerce platform with: PaymentComponent (Stripe API wrapper), InventoryComponent (Product catalog manager).

An architecture using inheritance and interfaces to enable interchangeable components without modifying core logic. Example: A payment processing system where CreditCardPayment and PayPayPayment implement a common Ipayment interface.

An architecture applying OOP principles (e.g., encapsulation) to enforce security (e.g., access control, input validation). Example: A User class with private password fields and public authentication() methods to prevent direct data exposure.